

Deployment Strategy

AI Schema Builder

Deployment Strategy & Operations Guide

Version 1.0 | 2025

1. Deployment Overview

This document defines the deployment strategy for the AI Schema Builder application across three environments: Development, Staging, and Production. The strategy follows a containerized microservices approach using Docker and orchestrated via Kubernetes (AWS EKS) or AWS ECS Fargate.

Environment

Purpose

Platform

Access

Development

Local dev & feature testing

Docker Compose

Engineers only

Staging

Pre-release validation & QA

AWS ECS Fargate

Internal team

Production

Live end-user environment

AWS EKS (Kubernetes)

Public

2. Infrastructure Architecture

2.1 AWS Production Architecture

Route 53 – DNS with health checks and failover routing

CloudFront CDN – caches static frontend assets globally

Application Load Balancer (ALB) – distributes traffic to backend services

EKS (Elastic Kubernetes Service) – backend API pod orchestration

RDS PostgreSQL (Multi-AZ) – primary database with automated failover

ElastiCache Redis – session store and API response caching

S3 – static frontend hosting and schema export file storage

Secrets Manager – encrypted storage for Gemini API keys and DB credentials

CloudWatch – metrics, logs, alarms, and dashboards

2.2 Container Architecture

Each service is packaged as a Docker container. Images are tagged with the Git commit SHA and pushed to Amazon ECR (Elastic Container Registry).

Service

Image

Port

Replicas (Prod)

Frontend

schema-builder/frontend:latest

3000 (internal)

2

Backend API

schema-builder/api:latest

8080

3

Background Worker

`schema-builder/worker:latest`

N/A

1

Nginx Proxy

`nginx:alpine`

80/443

2

3. CI/CD Pipeline

3.1 Pipeline Stages

Stage

Trigger

Actions

Approvals

Code Push

`PR to main`

Lint, Unit Tests, Build

Auto

Integration

PR merged

Integration tests, Docker build

Auto

Staging Deploy

Tag: staging/*

Push to ECR, Deploy to ECS

Auto

Prod Deploy

Tag: v*.*.*

Push to ECR, Deploy to EKS

Manual approval

Post-Deploy

After deploy

Smoke tests, Rollback if fail

Auto

3.2 GitHub Actions Workflow

The following key pipeline steps are executed on every deployment:

1. Checkout code

2. Run: npm test && npm run build

3. docker build -t \$ECR_REPO:\$GIT_SHA .

4. docker push \$ECR_REPO:\$GIT_SHA

5. kubectl set image deployment/api api=\$ECR_REPO:\$GIT_SHA

6. kubectl rollout status deployment/api

7. Run smoke tests → rollback on failure

4. Environment Configuration

4.1 Environment Variables

Variable

Description

Source

DATABASE_URL

PostgreSQL connection string

AWS Secrets Manager

GEMINI_API_KEY_ENCRYPTION_KEY

AES-256 key for Gemini API key encryption

AWS Secrets Manager

JWT_SECRET

JWT signing secret

AWS Secrets Manager

REDIS_URL

ElastiCache Redis endpoint

ECS/EKS config

NODE_ENV

Application environment

ECS/EKS config

CORS_ORIGIN

Allowed frontend origin URL

ECS/EKS config

PORT

API server port (default: 8080)

ECS/EKS config

4.2 Docker Compose (Development)

services:

frontend:

build: ./frontend

ports: ['3000:3000']

volumes: ['./frontend:/app']

api:

build: ./backend

ports: ['8080:8080']

env_file: .env.development

depends_on: [postgres, redis]

postgres:

image: postgres:15-alpine

environment:

POSTGRES_DB: schemabuilder

redis:

image: redis:7-alpine

5. Database Strategy

5.1 Migrations

Database migrations are managed using Knex.js migrations. Migrations run automatically at application startup in development and are run as a pre-deploy step (Kubernetes Job) in production.

5.2 Backup & Recovery

RDS automated backups – 7-day retention with point-in-time recovery

Daily snapshots exported to S3 (cross-region for disaster recovery)

Recovery Time Objective (RTO): < 1 hour

Recovery Point Objective (RPO): < 15 minutes

6. Scaling Strategy

Component

Scaling Type

Trigger

Min/Max

Backend API

Horizontal Pod Autoscaler

CPU > 70% or memory > 80%

2 / 10 pods

Background Worker

Job queue-based scaling

Queue depth > 100 tasks

1 / 5 pods

RDS Postgres

Vertical (instance resize)

Manual / scheduled maintenance

db.t3.medium → db.r6g.large

ElastiCache Redis

Cluster mode

Memory > 75%

1 / 3 nodes

7. Monitoring & Alerting

7.1 Key Metrics

API response time (p50, p95, p99) – alert if p95 > 2s

Error rate – alert if > 1% of requests return 5xx

Gemini API call success rate – alert if < 98%

Database connection pool utilization – alert if > 80%

Pod/container restart count – alert on > 3 restarts/hour

7.2 Alerting Channels

PagerDuty – for P0/P1 production incidents

Slack #alerts – for warnings and non-critical events

Email – daily digest of system health metrics

End of Deployment Strategy Document